

SIH26231 – Consolidated Test Flow: Replace Demo Logic With Real Processing

Hand this to your coder as one sequential build. It assumes camera + GPS capture already work – it does not rebuild them, it wires their output into real logic.

Step 0 – Remove the demo logic first

Find wherever the app currently generates a Positive/Negative result (likely a manually toggled boolean, a hardcoded value, or a random/mock function). Delete this entirely – do not leave it as a fallback or "test mode" switch, because a leftover fake-result path is exactly the kind of thing that ends up firing accidentally in a demo. The result must come from Steps 3-4 below, with no other code path capable of producing a result.

Step 1 – "Start Test" → Kit Selection screen (the 3 columns)

Build a KitSelectionScreen shown immediately after tapping "Start Test" on the Home screen. Three selectable cards, equal width, stacked vertically (not horizontal columns – on a phone width, 3 side-by-side cards would violate the 48dp minimum touch target from the design system):

Card 1 – "NDDK" – Narcotic Drugs Detection Kit – short subtitle: "Opiates, cannabis, cocaine, amphetamines"
Card 2 – "PCDK" – Precursor Chemicals Detection Kit – short subtitle: "Precursor chemicals used in synthesis"
Card 3 – "KDK" – Ketamine Detection Kit – short subtitle: "Ketamine"

Each card: white surface (#FFFFFF), 16dp padding, 16dp corner radius, icon + title + subtitle, full tap target on the whole card (not just the title text). Selecting a card sets a KitType enum value and navigates to the Reference Card Scan screen (already specified in the earlier build-prompts file), passing the selected KitType forward through the rest of the flow – every following screen

needs this value to know which reagent sequence to run.

Step 2 – Reference card / slip QR scan

Already fully specified in

SIH26231_App_Design_Build_Prompts.md Section 5.3 – use that prompt as-is. The only addition:

The scanned card's QR payload should also encode or be cross-checked against the KitType selected in Step 1 – if the officer selected NDDK but scans a card provisioned for KDK, show a clear mismatch warning and block proceeding until resolved. This stops an officer accidentally running the wrong reagent sequence against the wrong card.

Step 3 – Capture (reuse your existing working camera/GPS code)

Do NOT rebuild the camera or GPS capture – they already work. Just make sure

the CaptureScreen, when it finishes capturing the burst of frames and the GPS coordinate, hands both off to the processing pipeline in Step 4 along with the KitType from Step 1 and the card serial from Step 2. If your existing capture code returns these as separate variables/callbacks right now, wrap them into a single object (e.g. a CaptureResult class: frames, gpsPoint, locationConfirmed, kitType, cardSerial, reactionTimestamp) so Step 4 has one clean input.

Step 4 – Real OpenCV processing (this replaces the demo toggle)

Build the actual processing pipeline described in SIH26231_Scanning_Logic_CloudSync_Prompts.md, Section 2 (multi-step classification engine). Concretely, for THIS integration:

1. Load the reagent sequence config (Section 1 of that file) filtered to the KitType passed in from Step 1.
2. Run real ArUco marker detection on the captured frame to find the reference

card, then compute white-balance correction from its known patches – this must run against actual pixel data from the captured frame, never a hardcoded/stubbed color value.

3. Convert the corrected reaction-zone color to CIE Lab, compute ΔE_{2000} against every candidate outcome for this kit+step in the config, select the closest match under threshold.
4. If the matched outcome requires a follow-up reagent step (e.g. NDDK's Marquis → Nitric acid sequence), show the StepIndicator UI prompting the officer to apply the next reagent, then repeat capture+classify for that step.
5. Confidence % shown to the officer = a real computed value combining (a) how close the ΔE match was to the reference vs. the threshold margin, and (b) the image quality score from the capture-time checks (blur/exposure/card detection) – not a fixed or random number. Show your coder this exact formula requirement: confidence must visibly change if you test with two different quality photos of the same reaction – if it doesn't change, the computation isn't real.
6. Output the same ClassificationResult object already specified (category, confidence, interferentWarnings) and pass it directly to the Result screen.

Verify this works by testing with at least two genuinely different captured images (not the same image twice) and confirming the result and confidence differ appropriately – this is your own proof it isn't still a demo.

Step 5 – Result screen

Already fully specified in

`SIH26231_App_Design_Build_Prompts.md` Section 5.5 – use as-is. It already expects exactly the `ClassificationResult` object Step 4 now produces for real.

Step 6 – Local save, then cloud sync (for your website to read)

On "Confirm & Record," save the finalized `TestRecord` locally first (hash-chain append + dual signature, as already specified), then hand it to the `SyncService` from `SIH26231_Scanning_Logic_CloudSync_Prompts.md` Section 3.

This is what your separate website will

read from: the SyncService writes record metadata to Firestore (collection: "tests", document ID = testId) and images to Firebase Storage (path: records/{testId}/frame_{n}.jpg). Your website can query Firestore directly with the Firebase JS SDK using the same project – no separate backend API is required for the website to "seek the data," it reads the same Firestore/Storage your app writes to, subject to the security rules already specified (officers write their own records, your website's admin/auditor view reads with broader but still role-gated access).

Confirm with your web team: they should build against this exact Firestore schema (Step 3's field list in the cloud-sync prompts file) rather than inventing a separate one, or the two sides won't line up.

This file is the connective tissue between what you've already built (camera/GPS/UI) and the two prior logic files. Build Steps 1 → 6 in order – each depends on the previous one's output.